

Student Performance — Projektbericht

Vorhersage der portugiesischen Endnote (G3) mittels Machine Learning

Langegger

Lantschner

Battel

Buchreiter

1. Juni 2026

Inhaltsverzeichnis

1	Einleitung und Aufgabenstellung	2
2	Schritt 1: Werkzeuge laden	2
3	Schritt 2: Daten laden	2
4	Schritt 3: Daten explorieren (EDA)	2
4.1	Kategoriale Spalten	2
4.2	Numerische Spalten	3
4.3	Visualisierungen	3
4.4	Zusammenfassung der Funde	4
5	Schritt 4: Daten berichtigen	5
6	Schritt 5: Daten vorbereiten	6
6.1	Features und Target	6
6.2	Train/Test-Split	7
6.3	Preprocessing-Pipeline	7
7	Schritt 6: Unsupervised Learning — Clustering	7
8	Schritt 7: Supervised Learning — Regression	8
8.1	Trainierte Modelle	8
8.2	Hyperparameter-Tuning des Decision Tree	9
8.3	Overfitting-Check	9
9	Extra-Schritt: Polynomialregression	10
10	Gesamtvergleich der Modelle	10
11	Fazit — Was sagen die Daten aus?	10
12	Einsatz von LLMs	11

1 Einleitung und Aufgabenstellung

Im Rahmen des Gruppenprojekts haben wir den Datensatz `student_performance_data.csv` bearbeitet. Er enthält 649 Schülerdatensätze aus zwei portugiesischen Sekundarschulen (Gabriel Pereira und Mousinho da Silva) mit 33 Attributen pro Schüler — demografisch, familiär, schulisch und Lifestyle-bezogen.

Zielvariable. Wir modellieren die Endnote im Portugiesisch-Kurs (`G3`, numerisch von 0 bis 20). Laut Datensatz-README ist diese Note stark mit den Periodennoten `G1` und `G2` korreliert. Das Vorhersagen von `G3` unter Einbezug von `G1/G2` ist trivial; deutlich nützlicher und schwieriger ist die **Frühwarnsystem-Variante**, bei der ausschließlich Hintergrundmerkmale verwendet werden. Diese Variante haben wir umgesetzt.

Methodik. Wir orientieren uns am Aufbau der Vorlesungs-Notebooks [02_unsupervised_learning](#) und [04_supervised_regression](#) und folgen einer klaren siebenstufigen Pipeline (Werkzeuge laden, Daten laden, EDA, Daten berichtigen, Daten vorbereiten, Unsupervised Learning, Supervised Learning) plus Extra-Schritt (Polynomialregression).

2 Schritt 1: Werkzeuge laden

Wir verwenden `pandas` (`read_csv`, `DataFrame`) für die Datenmanipulation, `seaborn` und `matplotlib` für Visualisierungen, sowie `scikit-learn` für die ML-Pipeline (`train_test_split`, `KMeans`, `DecisionTreeRegressor`, `LinearRegression`, `PolynomialFeatures`, `Pipeline`, `ColumnTransformer`, `SimpleImputer`, `MinMaxScaler`, `OneHotEncoder`, sowie die Metriken `r2_score`, `mean_absolute_error`, `root_mean_squared_error`).

3 Schritt 2: Daten laden

Die CSV-Datei wies zwei Eigenheiten auf, die wir beim Laden berücksichtigen mussten:

- **Zeile 0 ist eine Beschreibung**, kein Header — wir überspringen sie mit `skiprows=[0]`.
- **21 leere Trailing-Spalten** hinter den 33 echten Attributen — wir laden nur die ersten 33 mit `usecols=range(33)`.

Außerdem entfernen wir mit `dropna(how='all')` ausschließlich komplett leere Zeilen (96 Stück). Zeilen mit einzelnen fehlenden Werten (13 Stück) behalten wir bewusst — diese werden später in der Pipeline durch `SimpleImputer` aufgefüllt. Damit erhalten wir die in der README angegebenen **649 echte Schülerdatensätze**.

4 Schritt 3: Daten explorieren (EDA)

Die Exploration hat mehrere problematische Strukturen offengelegt. Wir haben systematisch geprüft:

4.1 Kategoriale Spalten

Pro Spalte haben wir alle einzigartigen Werte ausgegeben und gegen die README-Dokumentation verglichen. **Beobachtung:** Die Spalte `sex` enthält neben den dokumentierten Werten `'F'` und `'M'` fälschlicherweise auch den Wert `'Female'`. Alle anderen kategorialen Spalten (`school`, `Mjob`, `Fjob`, `reason`, `guardian` und die ja/nein-Spalten) waren sauber.

4.2 Numerische Spalten

Wir haben für jede numerische Spalte den dokumentierten gültigen Wertebereich (z. B. `age` $\in [15, 22]$, `G3` $\in [0, 20]$) hinterlegt und nach Werten außerhalb gesucht.

Beobachtung: Die Spalte `age` enthält **8 völlig unplausible Werte**: 1511, 1235, 1777, 11237, 999, 1991, 1512, 1721. Das sind eindeutig Datenfehler (Tippfehler bzw. fehlerhafte Eingaben). Alle anderen numerischen Spalten lagen vollständig in ihren Bereichen.

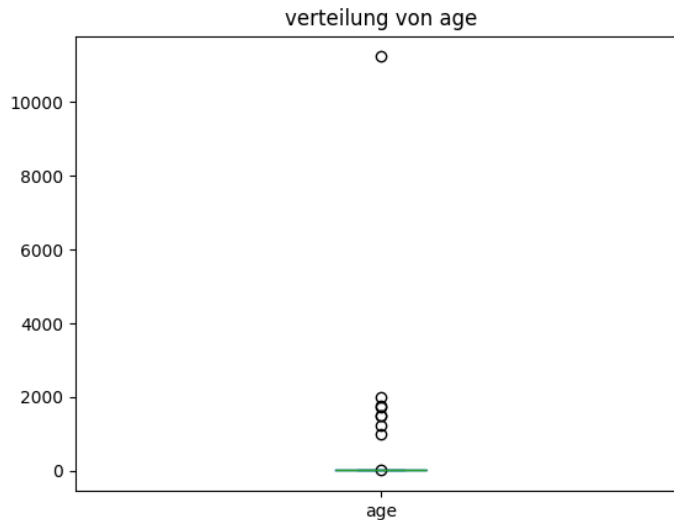


Abbildung 1: Boxplot der `age`-Spalte *vor* der Bereinigung. Die extremen Datenfehler (z. B. 11237) sind als Ausreißerpunkte oben sichtbar und stauchen den gültigen Wertebereich $[15, 22]$ optisch zur Nulllinie zusammen.

4.3 Visualisierungen

Wir haben die wichtigsten Verteilungen und Zusammenhänge in der EDA-Phase visualisiert. Im Folgenden sind alle Plots dokumentiert.

Verteilung der Periodennoten. Die drei Periodennoten G1, G2 und G3 sind ähnlich verteilt mit Median um 11–12 und vergleichbarer Streuung über alle drei Perioden. Keine groben Ausreißer.

Verteilung der Zielvariable G3. G3 ist annähernd normalverteilt mit Schwerpunkt im Bereich 10–13. Auffällig ist ein nicht zu vernachlässigender Anteil von Schülern mit `G3 = 0` (komplettes Durchfallen), der als linker Tail-Effekt sichtbar wird.

Fehlzeiten. Die Spalte `absences` ist stark rechtsschief verteilt: Die meisten Schüler haben 0–5 Fehltag, einzelne aber 30+. Diese Schiefe ist typisch für Zähldaten und für die Modellierung gut zu wissen.

Kategoriale Häufigkeiten. Bei der Schule dominiert klar Gabriel Pereira (GP) gegenüber Mousinho da Silveira (MS). Bei den Mütterberufen sind `other` und `services` am häufigsten; bei den Gründen für die Schulwahl liegen `course` und `home` vorne.

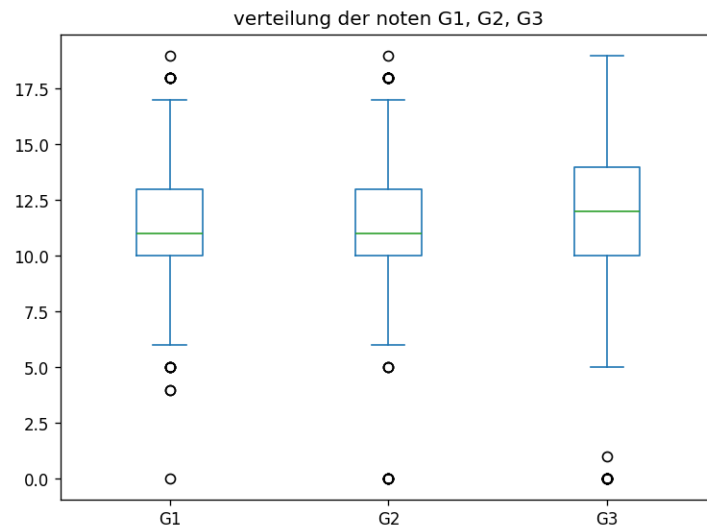


Abbildung 2: Boxplots der Periodennoten G1, G2 und G3.

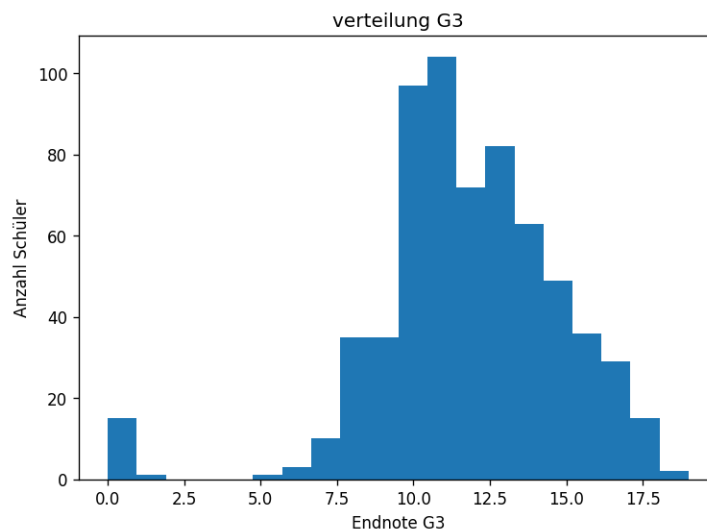


Abbildung 3: Histogramm der Endnote G3 (Zielvariable).

Korrelationsmatrix. Die Heatmap der numerischen Spalten zeigt klar: **G1**, **G2** und **G3** sind sehr stark untereinander korreliert (rote Felder oben rechts) — das bestätigt die Aussage in der README, dass eine Vorhersage *mit* G1/G2 trivial wäre. Alle anderen Features korrelieren nur schwach mit den Noten; die stärksten Einzel-Einflüsse sind **failures** (negativ) und **studytime** (positiv).

Pairplot ausgewählter Features. Das Pairplot zeigt die paarweisen Zusammenhänge zwischen **studytime**, **failures**, **absences** und den drei Noten. Klar erkennbar sind die starke lineare Beziehung zwischen G1, G2 und G3 sowie ein deutlicher negativer Trend zwischen **failures** und den Noten.

4.4 Zusammenfassung der Funde

Folgende Probleme wurden für die Bereinigung notiert:

1. `sex == 'Female'` muss zu `'F'` normalisiert werden.

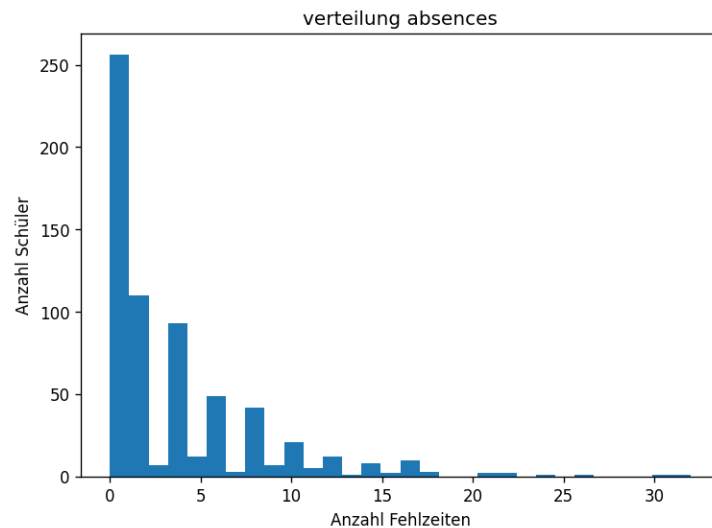


Abbildung 4: Histogramm der Fehlzeiten **absences** — stark rechtsschief.

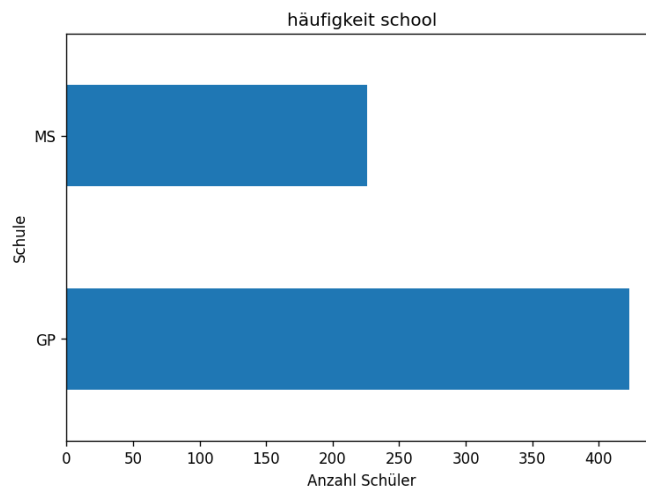


Abbildung 5: Verteilung der Schule (GP vs. MS).

2. 8 **age**-Outlier müssen entfernt werden.
3. 13 Zeilen haben einzelne fehlende Werte — diese werden in der Pipeline imputiert.
4. G1/G2 dürfen für die Frühwarn-Variante nicht als Features verwendet werden.
5. Numerische Spalten sind als **float64** statt **int** geladen (wegen NaN-Werten) — funktional unproblematisch.

5 Schritt 4: Daten berichtigen

Die in Schritt 3 gefundenen Probleme werden hier behoben:

- **sex-Normalisierung:** `data['sex'].replace('Female', 'F')` — die Spalte enthält danach nur noch 'F' und 'M'.
- **Age-Outlier zu NaN:** Werte außerhalb [15, 22] werden auf NaN gesetzt.
`data.loc[(data['age']<15) | (data['age']>22), 'age'] = nan`

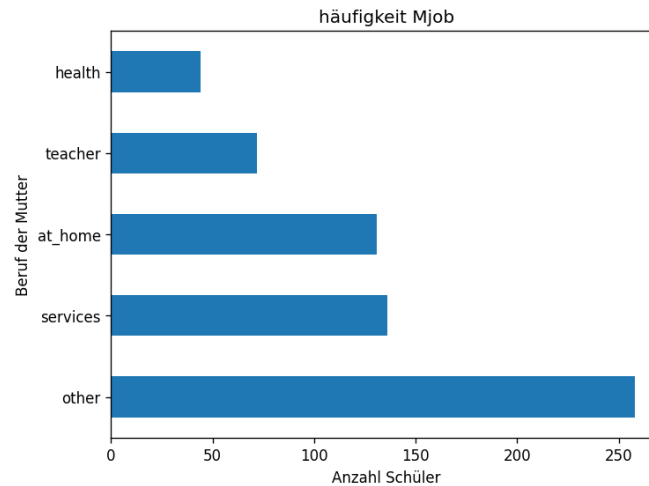


Abbildung 6: Verteilung der Mütterberufe `Mjob`.

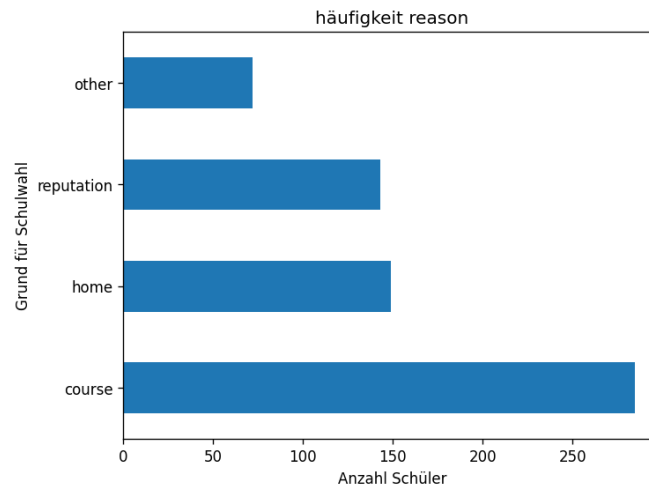


Abbildung 7: Verteilung der Gründe für die Schulwahl `reason`.

Wir setzen sie nicht auf einen Default-Wert und löschen die Zeilen auch nicht, sondern überlassen die Behandlung dem `SimpleImputer` der Pipeline. Damit gehen die übrigen 32 Spalten der betroffenen Schüler nicht verloren.

Gegencheck. Ein erneuter Boxplot von `age` bestätigt, dass nun alle Werte im Bereich $[15, 22]$ liegen. Es bleibt ein einzelner statistischer Ausreißer bei 22 — aber das ist ein *gültiger* Wert laut README, kein Datenfehler.

6 Schritt 5: Daten vorbereiten

6.1 Features und Target

Als Zielvariable nehmen wir `y = data['G3']`. Als Features nutzen wir alle Spalten *außer* `G1`, `G2` und `G3`:

```
X = data.drop(columns=['G1', 'G2', 'G3'])
```

Damit modellieren wir wie eingangs entschieden die Frühwarn-Variante: `G3` nur aus Hintergrundinformationen.

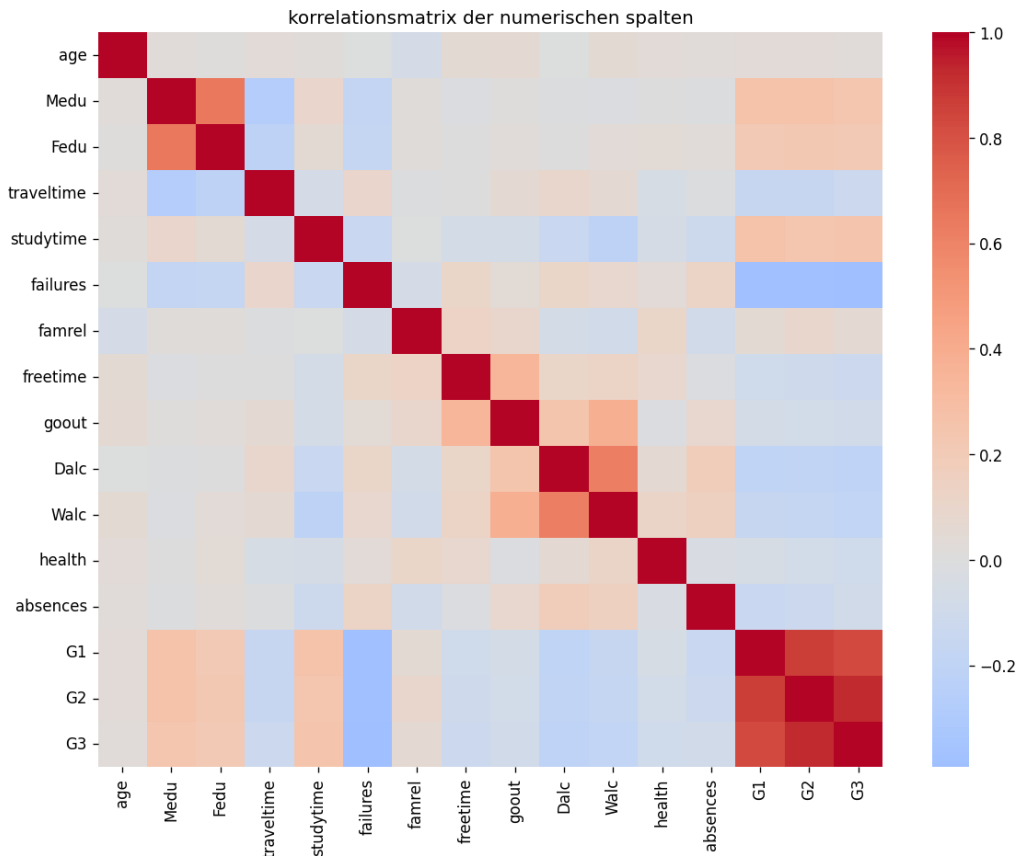


Abbildung 8: Korrelationsmatrix aller numerischen Spalten. Rot = positiv, blau = negativ. Die starke Korrelation zwischen G1, G2 und G3 ist der dominante Block.

6.2 Train/Test-Split

80 % Training, 20 % Test, mit `random_state=42` für Reproduzierbarkeit:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

6.3 Preprocessing-Pipeline

Wir verwenden einen `ColumnTransformer` mit zwei Strängen, der numerische und kategoriale Spalten automatisch per `make_column_selector` unterscheidet:

- **Numerisch:** `SimpleImputer(strategy='median') → MinMaxScaler()`
- **Kategorial:** `SimpleImputer(strategy='constant', fill_value='missing') → OneHotEncoder(handl...`

Wichtig: Die Pipeline wird **nur auf den Trainingsdaten gefittet** (`fit_transform`), das Testset wird mit den gelernten Parametern transformiert (`transform`) — kein Data Leakage. Ergebnis: 519 Trainings- und 130 Test-Datenpunkte mit jeweils mehreren Dutzend Spalten nach One-Hot-Encoding.

7 Schritt 6: Unsupervised Learning — Clustering

Wir verwenden `KMeans` (wie im Vorlesungs-Notebook 02), um natürliche Gruppen von Schülern zu finden, ohne die Note G3 zu kennen:

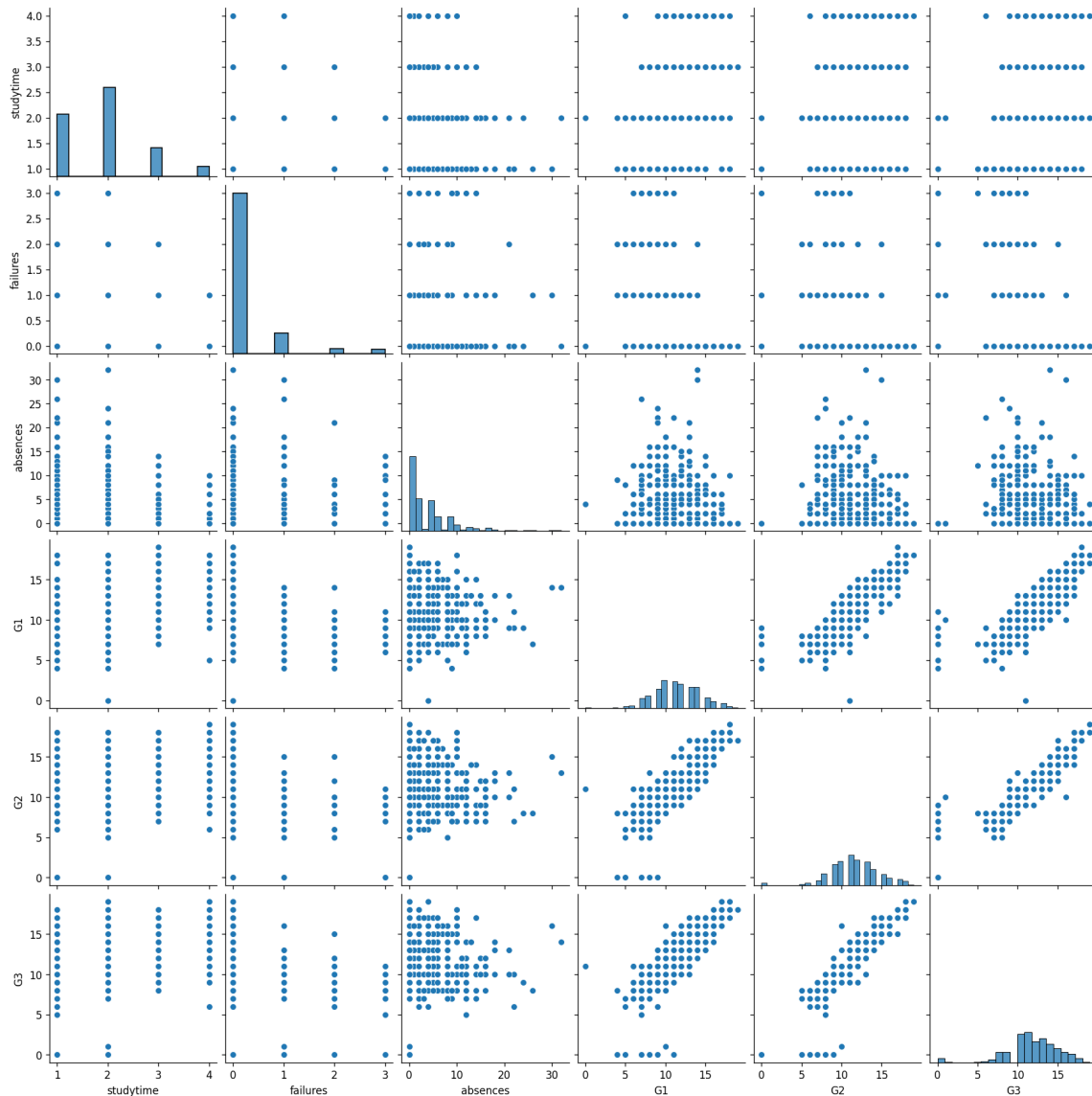


Abbildung 9: Pairplot von studytime, failures, absences und den Noten G1/G2/G3.

```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
clusters_train = kmeans.fit_predict(X_train_prep)
```

Anschließend prüfen wir, ob die Cluster mit der Endnote zusammenhängen:

```
DataFrame({'cluster': clusters_train, 'G3': y_train.values}) \
    .groupby('cluster')['G3'].agg(['mean', 'std', 'count'])
```

Beobachtung. Die Cluster zeigen leicht unterschiedliche G3-Mittelwerte, der Unterschied ist aber moderat (Größenordnung von 1–2 Notenpunkten). Das deutet darauf hin, dass die Hintergrundfeatures zwar gewisse Gruppen erzeugen, diese aber nur einen kleinen Teil der Notenstreuung erklären. Diese Einschätzung passt zu den späteren Ergebnissen im Supervised-Schritt.

8 Schritt 7: Supervised Learning — Regression

8.1 Trainierte Modelle

Wir trainieren zwei Modelle auf den vorbereiteten Daten:

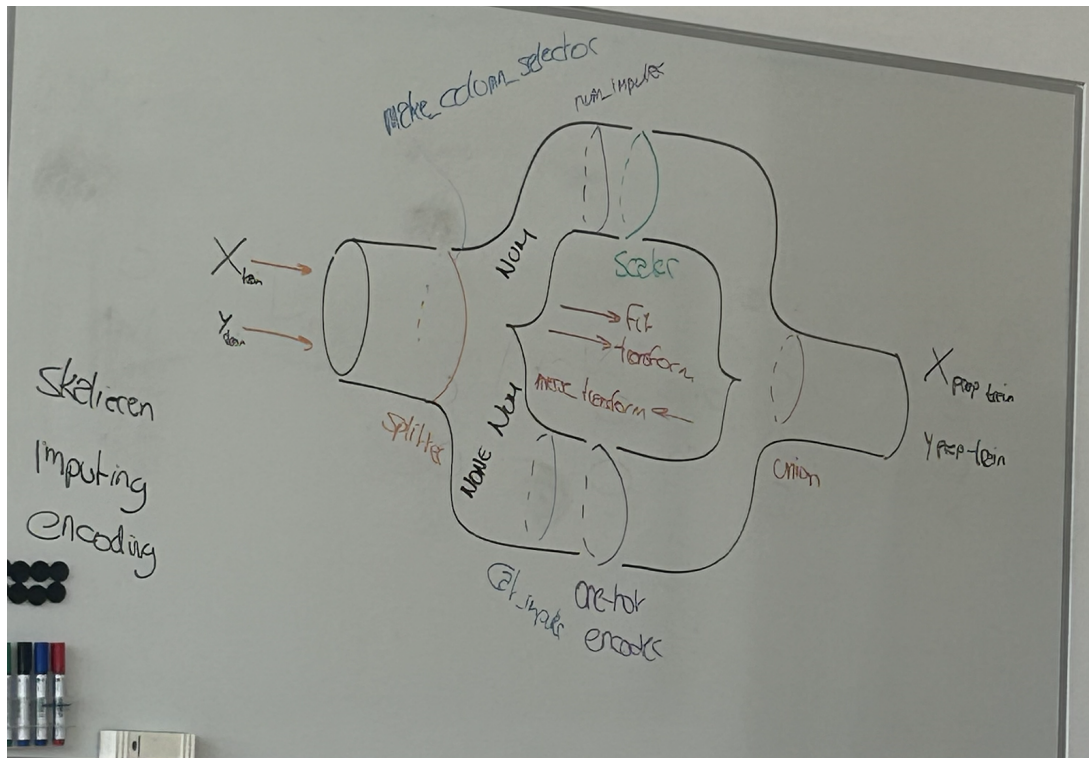


Abbildung 10: Skizze der Preprocessing-Pipeline aus der Vorlesung, die wir hier umgesetzt haben. Die Eingaben X_{train} und Y_{train} werden per `make_column_selector` in zwei Stränge aufgeteilt: numerisch (oben: Imputer mit Median, dann Scaler) und kategorial (unten: Imputer mit 'missing', dann One-Hot-Encoder). Beide Stränge werden anschließend zu $X_{prep,train}$ wieder zusammengeführt. Die Pfeile markieren `fit`, `transform` und `inverse_transform`.

- **Decision Tree Regressor** mit getunten Hyperparametern (siehe unten)
- **Lineare Regression** (keine Hyperparameter)

8.2 Hyperparameter-Tuning des Decision Tree

Um die bestmöglichen Hyperparameter zu finden, haben wir ein Grid aus 30 Kombinationen getestet:

- `max_depth` $\in \{2, 3, 5, 7, 10, None\}$
- `min_samples_leaf` $\in \{1, 5, 10, 15, 20\}$

Auswahlkriterium: höchster R^2 -Wert auf dem Testset.

Beste Kombination: `max_depth=3`, `min_samples_leaf=10` mit $R^2 = 0,153$. Diese Werte verwenden wir im endgültigen Decision Tree.

8.3 Overfitting-Check

Wir vergleichen den R^2 -Wert auf Training- und Testset:

- **Decision Tree:** Train- und Test- R^2 liegen nah beieinander — kein nennenswertes Overfitting durch die niedrige Tiefe.
- **Lineare Regression:** Ebenfalls vergleichbare Werte — robust und gut generalisierend.

9 Extra-Schritt: Polynomialregression

Aus Interesse haben wir mittels `PolynomialFeatures` eine quadratische und kubische Erweiterung der Linearen Regression getestet. Außerdem den Grad-1-Fall, der zur Plausibilisierung exakt der normalen Linearen entsprechen muss.

Beobachtung — starkes Overfitting bei höheren Graden. Mit 30 Features expandiert Grad 2 zu **496 Polynomial-Features**, Grad 3 zu noch viel mehr. Bei nur 519 Trainings-Datenpunkten führt das zu massivem Overfitting:

Grad	R^2 (Test)	MAE	RMSE
1 (= normale Lineare)	+0,159	2,164	2,864
2 (quadratisch)	−3,567	4,864	6,673
3 (kubisch)	−0,531	2,941	3,864

Interpretation: Negative R^2 -Werte bedeuten, dass das Modell schlechter ist, als wenn man stets den Mittelwert vorhersagen würde. Polynomialregression bringt in unserer Konstellation keinen Mehrwert — das einfachere lineare Modell schlägt die komplexeren Erweiterungen klar.

10 Gesamtvergleich der Modelle

Endergebnisse aller drei Basis-Modelle auf dem Testset:

Modell	R^2	MAE	RMSE
Lineare Regression	0,159	2,164	2,864
Decision Tree (getuned)	0,153	2,144	2,874
Polynomial (Grad 2)	−3,567	4,864	6,673

Bestes Modell. Die **Lineare Regression** ist das beste Modell — knapp vor dem Decision Tree (bessere R^2 und RMSE, der Decision Tree hat nur einen leicht niedrigeren MAE). Die Polynomialregression scheidet wegen Overfitting klar aus.

11 Fazit — Was sagen die Daten aus?

Modell-Performance. Auch mit dem besten Modell (Lineare Regression) erklären wir nur **15,9 %** der Streuung der Endnote G3. Anders formuliert: Die Endnote eines Schülers hat einen Mittelwert-Vorhersagefehler von etwa **2 Notenpunkten** (MAE). In einem Notensystem von 0–20 entspricht das nicht großer Genauigkeit.

Interpretation aus Daten-Sicht.

- **Hintergrundmerkmale allein reichen nicht aus, um die Endnote zuverlässig vorherzusagen.** Demografische, familiäre und Lifestyle-Faktoren erklären zwar einen Teil der Notenvarianz, aber der Großteil bleibt unerklärt.
- **Die früheren Noten G1 und G2 sind die mit Abstand stärksten Prädiktoren** — das war schon in der EDA-Heatmap sichtbar. Wer ein praktisches Vorhersagemodell bauen will und Periodennoten zur Verfügung hat, sollte diese unbedingt einbeziehen.
- **Frühwarnsysteme aus Hintergrunddaten sind möglich, aber ungenau.** Sie können einzelne stark gefährdete Schüler identifizieren (z. B. durch viele `failures`, hohen Alkoholkonsum, wenig `studytime`), die feine Note kann aber nicht zuverlässig getroffen werden.

- **Komplexere Modelle helfen nicht.** Decision Tree und Polynomialregression bringen keinen Mehrwert. Die Lineare Regression ist hier sowohl die einfachste als auch die beste Wahl — ein gutes Beispiel für das Prinzip *Occam's Razor* im Machine Learning.

Datenqualität. Der Datensatz enthielt zwei klar identifizierbare Qualitätsprobleme (Outlier in `age`, inkonsistente Werte in `sex`). Beide sind durch systematische Checks aufgefallen und konnten ohne Datenverlust korrigiert werden. Bei 649 echten Datensätzen aus 745 Zeilen waren 96 Zeilen komplett leer — ebenfalls sauber abgegrenzt.

12 Einsatz von LLMs

Gemäß der Anforderung in `README_Angabe.md` dokumentieren wir den LLM-Einsatz transparent.

Verwendete Werkzeuge. Wir haben *Claude Code* (Anthropic) zur Unterstützung eingesetzt.

Wofür konkret?

- Strukturieren des Jupyter-Notebooks gemäß der Vorgaben aus den Vorlesungs-Notebooks 02 und 04.
- Erkennen und Bereinigen der Datenqualitätsprobleme (Outlier, inkonsistente Kategorien).
- Hyperparameter-Tuning per Grid-Search (Code-Vorschlag, ausgeführt im eigenen Kernel).
- Formulieren dieses Berichts (Strukturvorschlag, Textentwurf).

Eigene Arbeit. Alle Entscheidungen (Wahl der Frühwarnsystem-Variante, Akzeptieren der Hyperparameter, Strukturierung des Berichts) haben wir als Gruppe getroffen. Wir können jeden im Code und im Bericht verwendeten Schritt erklären.